

Python pour le Machine Learning

Les Outils Essentiels

Meyssem Bouzaïen

Année universitaire: 2025 - 2026

Plan du Cours

- 1 Introduction et Environnement
- 2 NumPy - Calcul Numérique
- 3 Pandas - Manipulation de Données
- 4 Comprendre la Corrélation
- 5 Matplotlib - Visualisation
- 6 Scikit-learn - Premier Modèle ML
- 7 Workflow et Bonnes Pratiques

Ce que vous allez apprendre

- Maîtriser NumPy, Pandas, Matplotlib et Scikit-learn
- Manipuler et préparer des données réelles
- Visualiser vos résultats
- Construire votre premier modèle ML

Environnement de Travail : Vous avez le choix!

Options disponibles:

- Google Colab (recommandé)
- Jupyter Notebook
- Anaconda + Jupyter
- VS Code
- PyCharm

Utilisez l'environnement de votre choix!

Pourquoi Google Colab?

- Gratuit
- Aucune installation
- GPU disponible
- Facile à partager

Les 4 Bibliothèques Essentielles

NumPy

Calcul numérique

Pandas

Manipulation données

Matplotlib

Visualisation

Scikit-learn

Algorithmes ML

NumPy : C'est quoi exactement?

Définition

NumPy (Numerical Python) est la bibliothèque fondamentale pour le calcul scientifique en Python. Elle fournit des structures de données performantes pour manipuler des tableaux multidimensionnels.

Caractéristiques:

- Arrays N-dimensionnels
- Opérations vectorisées
- Optimisé en C
- Base pour Pandas, Scikit-learn

Types de données:

- int32, int64
- float32, float64
- bool, complex
- Homogènes dans un array

NumPy : Pourquoi?

Listes Python:

```
1 liste = [1, 2, 3, 4]
2 resultat = []
3 for x in liste:
4     resultat.append(x * 2)
5 # Lent!
```

Arrays NumPy:

```
1 import numpy as np
2 array = np.array([1,2,3,4])
3 resultat = array * 2
4 # Rapide!
```

Avantages NumPy

- 50-100x plus rapide que les listes
- Syntaxe simple
- Moins de mémoire

Array 1D vs 2D vs 3D - Visualisation

Array 1D

5	8	3	9
---	---	---	---

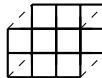
shape: (4,)

Array 2D (Matrice)

11	12	13	14	15
6	7	8	9	10
1	2	3	4	5

shape: (3, 5)

Array 3D



shape: (3, 3, 2)

Créer des Arrays

```
import numpy as np

# Depuis une liste
a = np.array([1, 2, 3, 4, 5])
print(a)  # [1 2 3 4 5]

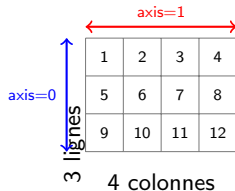
# Matrice 2D
m = np.array([[1, 2, 3],
              [4, 5, 6]])
print(m.shape)  # (2, 3)

# Arrays spéciaux
zeros = np.zeros((3, 4))          # Matrice de zeros
ones = np.ones((2, 3))            # Matrice de uns
range_arr = np.arange(0, 10, 2)   # [0 2 4 6 8]
linear = np.linspace(0, 1, 5)     # 5 valeurs [0, 0.25, 0.5, 0.75, 1]
random = np.random.rand(3, 3)     # Valeurs aleatoires [0, 1)
```

Dimensions et Formes (shape)

```
data = np.array([[1, 2, 3, 4],
                 [5, 6, 7, 8],
                 [9, 10, 11, 12]])

print(data.shape) # (3, 4)
print(data.ndim)  # 2
print(data.size)  # 12
print(data.dtype) # int64
```



Important

axis=0 : opérations sur les lignes (vertical)

axis=1 : opérations sur les colonnes (horizontal)

Comment NumPy opère sur des arrays de tailles différentes

Exemple 1: Array + Scalaire

$$\begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} + \boxed{5} = \begin{bmatrix} 6 & 7 & 8 & 9 \end{bmatrix}$$

Exemple 2: Matrice + Vecteur

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} + \begin{bmatrix} 10 & 20 & 30 \end{bmatrix} = \begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \\ 17 & 28 & 39 \end{bmatrix}$$

Opérations de base:

```
1 a = np.array([1,2,3,4])
2 b = np.array([10,20,30,40])
3
4 a + b # [11 22 33 44]
5 a * b # [10 40 90 160]
6 a ** 2 # [1 4 9 16]
7 a / 2 # [0.5 1 1.5 2]
```

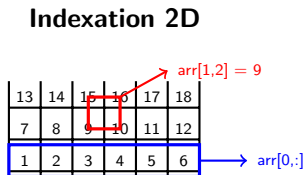
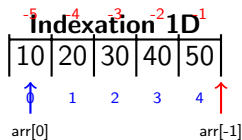
Fonctions utiles:

```
1 data = np.array([1,2,3,4,5])
2
3 np.mean(data) # 3.0
4 np.sum(data) # 15
5 np.min(data) # 1
6 np.max(data) # 5
7 np.std(data) # Ecart-type
8 np.sqrt(data) # Racines
9 np.median(data) # Mediane
```

Important

Toutes les opérations se font élément par élément!

Indexation - Visualisation



Indexation et Slicing

```
# Array 1D
arr = np.array([10, 20, 30, 40, 50])
arr[0]      # 10 - premier element
arr[-1]     # 50 - dernier element
arr[1:4]    # [20 30 40] - slicing

# Array 2D
mat = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

mat[0, 0]   # 1 - premiere ligne, premiere colonne
mat[1, :]   # [4 5 6] - toute la ligne 1
mat[:, 2]   # [3 6 9] - toute la colonne 2
mat[0:2, 1:3] # [[2 3]
               #  [5 6]] - sous-matrice
```

À vous de jouer!

- 1 Créez un array de 20 nombres aléatoires entre 0 et 100
- 2 Calculez la moyenne et l'écart-type
- 3 Trouvez les valeurs supérieures à 50
- 4 Créez une matrice 5x4 de nombres aléatoires
- 5 Calculez la somme de chaque colonne

```
# Indice pour commencer
import numpy as np

# Generer des nombres aleatoires
data = np.random.randint(0, 100, 20)
```

Pandas : C'est quoi?

DataFrame = Tableau Excel en Python

- Structure de données tabulaire
- Colonnes avec des noms
- Lignes avec des index
- Types de données mixtes

Nom	Age	Ville	Score
Alice	25	Paris	85
Bob	30	Lyon	92
Charlie	22	Marseille	78

C'est comme ça qu'on travaille avec des données en ML!

Créer un DataFrame (1/2)

```
import pandas as pd

# Methode 1 : Depuis un dictionnaire
data = {
    'Nom': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 22],
    'Ville': ['Paris', 'Lyon', 'Marseille'],
    'Score': [85, 92, 78]
}
df = pd.DataFrame(data)
print(df)
```

	Nom	Age	Ville	Score
0	Alice	25	Paris	85
1	Bob	30	Lyon	92
2	Charlie	22	Marseille	78

Créer un DataFrame (2/2)

Charger depuis un fichier CSV:

```
import pandas as pd

# Depuis un fichier CSV
df = pd.read_csv('donnees.csv')

# Depuis un fichier Excel
df = pd.read_excel('donnees.xlsx')

# Depuis une URL
url = 'https://example.com/data.csv'
df = pd.read_csv(url)
```

Premiers réflexes:

```
df.head()      # 5 premières lignes
df.tail()      # 5 dernières lignes
df.shape       # (lignes, colonnes)
df.info()      # Infos sur colonnes
df.describe()  # Statistiques
```

Explorer les Données

```
# Charger le dataset Iris
df = pd.read_csv('iris.csv')

# Dimensions
print(df.shape) # (150, 5)

# Noms des colonnes
print(df.columns)

# Types de donnees
print(df.dtypes)

# Statistiques descriptives
print(df.describe())

# Compter les valeurs uniques
print(df['species'].value_counts())

# Verifier les valeurs manquantes
print(df.isnull().sum())
```

Sélectionner des Données (1/2)

```
# Sélectionner une colonne
ages = df['Age']
# ou
ages = df.Age

# Sélectionner plusieurs colonnes
subset = df[['Nom', 'Score']]

# Filtrer les lignes
etudiants_reussis = df[df['Score'] > 80]

# Conditions multiples
filtre = df[(df['Age'] > 20) & (df['Score'] > 85)]
```

Sélectionner des Données (2/2)

```
# Sélectionner par position (iloc)
df.iloc[0]          # Première ligne
df.iloc[0:3]        # 3 premières lignes
df.iloc[0:3, 0:2]    # 3 lignes, 2 colonnes

# Sélectionner par label (loc)
df.loc[0, 'Nom']     # Valeur spécifique
df.loc[0:2, ['Nom', 'Age']] # Sélection mixte
```

iloc vs loc

- **iloc** : utilise les positions (0, 1, 2...)
- **loc** : utilise les labels/noms

Opérations sur les Données

```
# Ajouter une nouvelle colonne
df['Score_Double'] = df['Score'] * 2

# Modifier une colonne
df['Age'] = df['Age'] + 1

# Supprimer une colonne
df = df.drop('Score_Double', axis=1)

# Supprimer une ligne
df = df.drop(0, axis=0)

# Trier les données
df_trie = df.sort_values('Score', ascending=False)

# Grouper et agréger
moyennes = df.groupby('Ville')['Score'].mean()
```

Gérer les Valeurs Manquantes

```
# Verifier les valeurs manquantes
print(df.isnull().sum())

# Supprimer les lignes avec valeurs manquantes
df_clean = df.dropna()

# Supprimer les colonnes avec valeurs manquantes
df_clean = df.dropna(axis=1)

# Remplir avec une valeur
df_filled = df.fillna(0)

# Remplir avec la moyenne
df['Age'] = df['Age'].fillna(df['Age'].mean())

# Remplir avec la valeur precedente
df_filled = df.fillna(method='ffill')
```

Dataset Iris

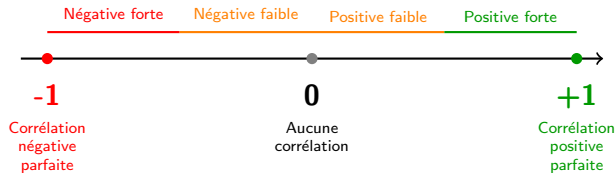
- 1 Chargez le dataset iris.csv
- 2 Affichez les 10 premières lignes
- 3 Combien y a-t-il de lignes et colonnes?
- 4 Quelles espèces sont présentes?
- 5 Calculez la moyenne de chaque colonne numérique
- 6 Filtrez les fleurs avec sepal_length $\geq$ 6
- 7 Créez: $\text{sepal_area} = \text{sepal_length} * \text{sepal_width}$

```
import pandas as pd  
# Votre code ici...
```

Qu'est-ce que la Corrélation? (1/3)

Définition

La corrélation mesure la **relation linéaire** entre deux variables quantitatives. Elle varie entre **-1** et **+1**.



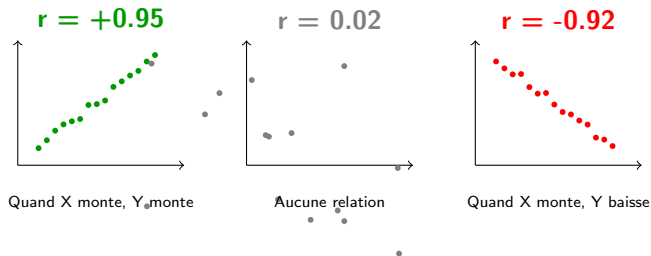
Qu'est-ce que la Corrélation? (2/3)

Interprétation des valeurs:

Valeur	Force	Signification
+0.90 à +1.00	Très forte +	Les deux augmentent ensemble
+0.70 à +0.89	Forte +	Relation claire
+0.40 à +0.69	Modérée +	Tendance visible
+0.10 à +0.39	Faible +	Relation légère
-0.09 à +0.09	Négligeable	Pas de relation
-0.10 à -0.39	Faible -	Relation inverse légère
-0.40 à -0.69	Modérée -	Tendance inverse
-0.70 à -0.89	Forte -	Relation inverse claire
-0.90 à -1.00	Très forte -	L'une monte, l'autre baisse

Qu'est-ce que la Corrélacion? (3/3)

Exemples visuels:



Important

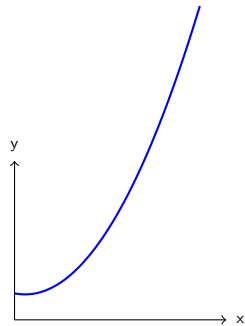
Corrélacion ne signifie PAS causalité!

Deux variables peuvent être corrélées sans que l'une cause l'autre.

Pourquoi Visualiser?

Un graphique vaut 1000 mots

- Comprendre les données
- Détecter les patterns
- Identifier les outliers
- Communiquer les résultats
- Vérifier les hypothèses



Tendance visible!

En ML

La visualisation est essentielle!

Premier Graphique

```
import matplotlib.pyplot as plt

# Donnees
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

# Creer le graphique
plt.plot(x, y)

# Personnaliser
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.title('Mon premier graphique')

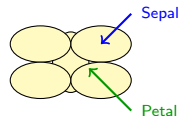
# Afficher
plt.show()
```

Simple et efficace!

Dataset Iris - Présentation

4 Features:

- sepal_length : Longueur sépale
- sepal_width : Largeur sépale
- petal_length : Longueur pétale
- petal_width : Largeur pétale



Fleur d'Iris

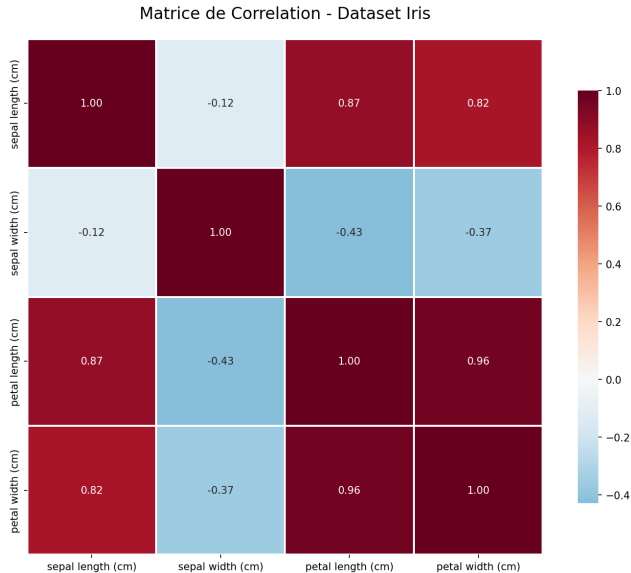
3 Espèces:

- Setosa
- Versicolor
- Virginica

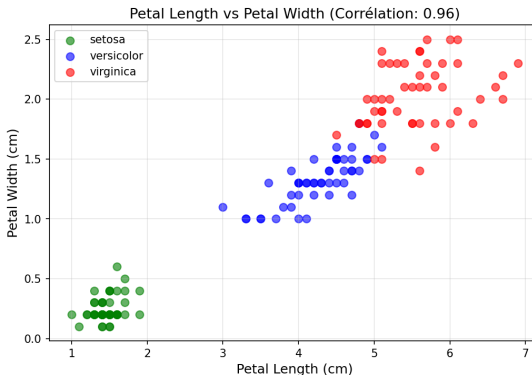
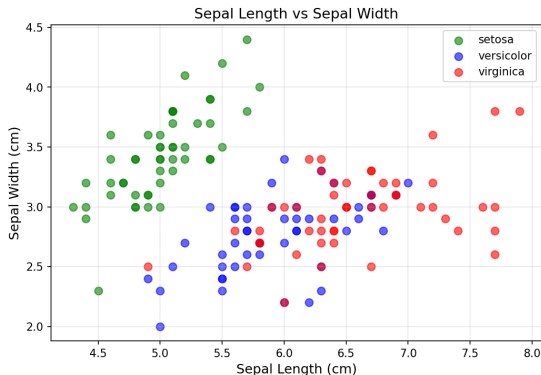
Objectif

Prédire l'espèce à partir des 4 mesures

Iris - Matrice de Corrélation



Iris - Relations Visuelles

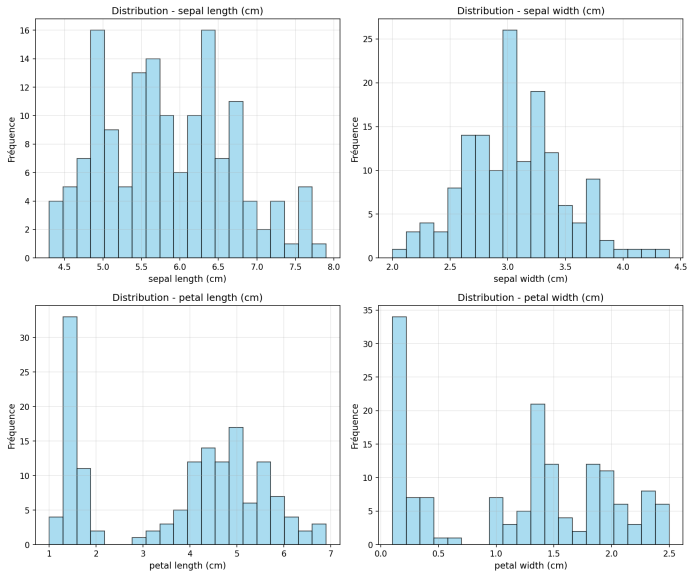


Qu'est-ce qu'on voit?

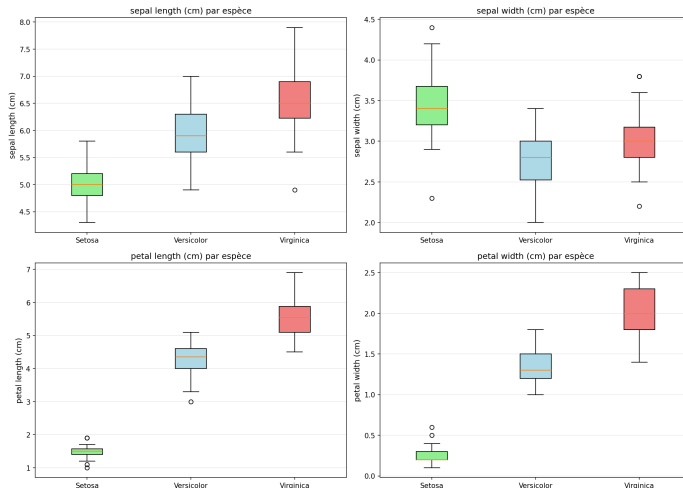
Gauche: Corrélation faible (-0.12) - points éparpillés

Droite: Corrélation très forte ($+0.96$) - alignement clair!

Iris - Histogrammes par Feature



Iris - Boxplots par Espèce



Observations

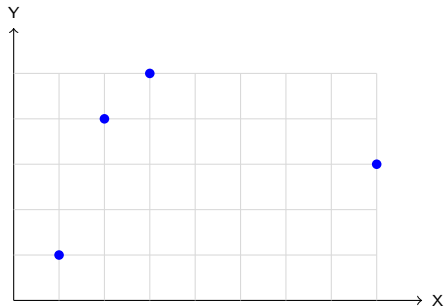
Les espèces sont bien séparées sur `petal_length` et `petal_width`!

Types de Graphiques - Nuage de points

Code:

```
1 import matplotlib.pyplot as plt
2
3 x = [1, 2, 3, 8]
4 y = [1, 4, 5, 3]
5
6 plt.scatter(x, y)
7 plt.xlabel('X')
8 plt.ylabel('Y')
9 plt.title('Nuage de points')
10 plt.show()
```

Résultat:

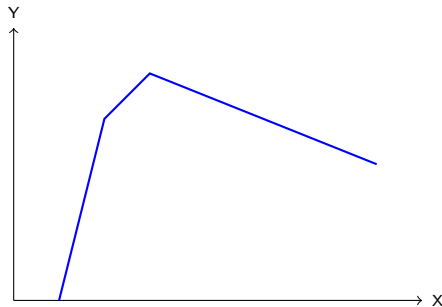


Types de Graphiques - Courbe

Code:

```
1 import matplotlib.pyplot as plt
2
3 x = [1, 2, 3, 8]
4 y = [0, 4, 5, 3]
5
6 plt.plot(x, y)
7 plt.xlabel('X')
8 plt.ylabel('Y')
9 plt.title('Courbe')
10 plt.show()
```

Résultat:

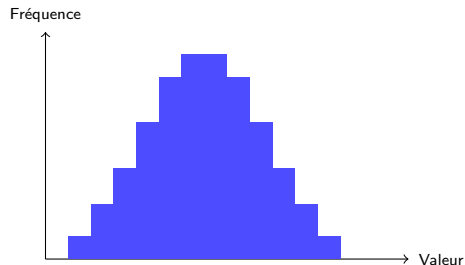


Types de Graphiques - Histogramme

Code:

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 data = np.random.randn(1000)
5
6 plt.hist(data, bins=20,
7         color='blue',
8         alpha=0.7)
9 plt.xlabel('Valeur')
10 plt.ylabel('Frequence')
11 plt.title('Histogramme')
12 plt.show()
```

Résultat:

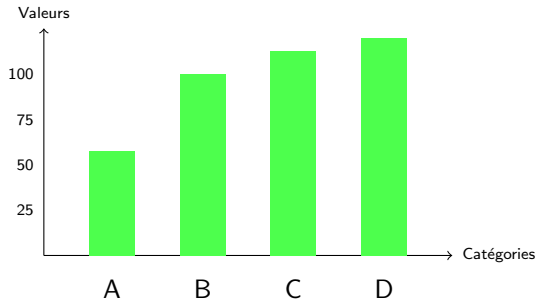


Types de Graphiques - Diagramme en barres

Code:

```
1 import matplotlib.pyplot as plt
2
3 categories = ['A', 'B', 'C', 'D']
4 valeurs = [23, 45, 56, 78]
5
6 plt.bar(categories, valeurs,
7         color='green')
8 plt.xlabel('Categories')
9 plt.ylabel('Valeurs')
10 plt.title('Diagramme en barres')
11 plt.show()
```

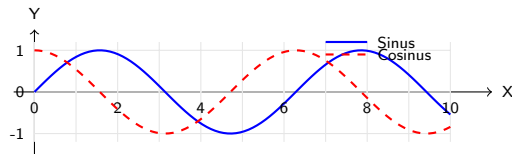
Résultat:



Plusieurs Courbes - Exemple Sinus/Cosinus

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(0, 10, 100)
5 y1 = np.sin(x)
6 y2 = np.cos(x)
7
8 plt.plot(x, y1, label='Sinus',
9          color='blue', linewidth=2)
10 plt.plot(x, y2, label='Cosinus',
11          color='red', linestyle='--',
12          linewidth=2)
13
14 plt.xlabel('X')
15 plt.ylabel('Y')
16 plt.title('Fonctions Trigo')
17 plt.legend()
18 plt.grid(True)
19 plt.show()
```

Résultat:



```
# Couleurs
plt.plot(x, y, color='red') # ou 'r', '#FF0000'

# Styles de ligne
plt.plot(x, y, linestyle='--') # '--', '-.', ':'

# Marqueurs
plt.scatter(x, y, marker='o') # 'o', 's', '^', '*'

# Taille
plt.plot(x, y, linewidth=3)
plt.scatter(x, y, s=100)

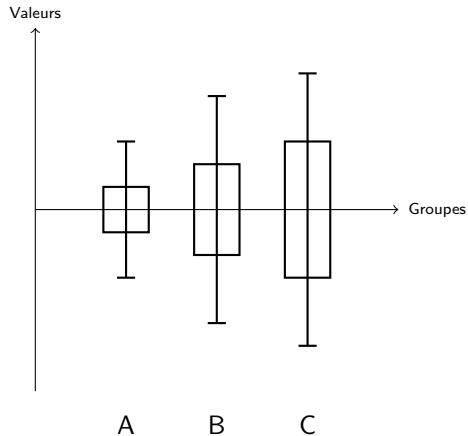
# Transparence
plt.scatter(x, y, alpha=0.5) # 0 a 1
```

Types de Graphiques - Boîte à moustaches

Code:

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 data = [np.random.normal(0, std,
5                          100) for std in
6         range(1, 4)]
7
8 plt.boxplot(data,
9             labels=['A', 'B', 'C'])
10 plt.xlabel('Groupes')
11 plt.ylabel('Valeurs')
12 plt.title('Boxplot')
13 plt.show()
```

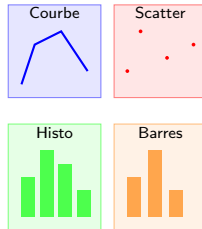
Résultat:



Sous-graphiques (Subplots) - Exemple

```
fig, axes = plt.subplots(2, 2,  
                          figsize=(10, 8))  
  
# Graphique 1  
axes[0, 0].plot(x, y1)  
axes[0, 0].set_title('Courbe')  
  
# Graphique 2  
axes[0, 1].scatter(x, y2)  
axes[0, 1].set_title('Scatter')  
  
# Graphique 3  
axes[1, 0].hist(data, bins=20)  
axes[1, 0].set_title('Histo')  
  
# Graphique 4  
axes[1, 1].bar(cat, val)  
axes[1, 1].set_title('Barres')  
  
plt.tight_layout()  
plt.show()
```

Résultat:



Visualiser avec Pandas

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('iris.csv')

# Histogramme d'une colonne
df['sepal_length'].hist(bins=20)
plt.show()

# Nuage de points
df.plot.scatter(x='sepal_length', y='sepal_width')
plt.show()

# Boite a moustaches par groupe
df.boxplot(column='petal_length', by='species')
plt.show()
```

Visualisez Iris

- 1 Histogramme pour sepal_length
- 2 Nuage de points: sepal_length vs sepal_width
- 3 Colorez par espèce
- 4 4 subplots: un histogramme par feature
- 5 Personnalisez avec titres, labels, couleurs

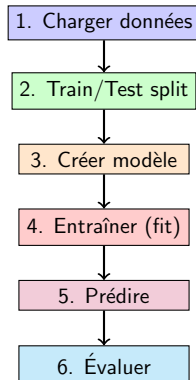
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('iris.csv')
# Votre code ici...
```

Tout pour le ML

- Algorithmes de classification
- Algorithmes de régression
- Algorithmes de clustering
- Prétraitement des données
- Évaluation des modèles
- Sélection de features

Interface simple et cohérente!



Classification Iris (1/3)

```
# 1. Imports
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 2. Charger les donnees
iris = load_iris()
X = iris.data    # Features (4 colonnes)
y = iris.target  # Labels (especes: 0, 1, 2)

print("X shape:", X.shape)  # (150, 4)
print("y shape:", y.shape)  # (150,)
```

Classification Iris (2/3)

```
# 3. Diviser en train/test (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Train:", X_train.shape)  # (120, 4)
print("Test:", X_test.shape)   # (30, 4)

# 4. Creer le modele
model = KNeighborsClassifier(n_neighbors=3)

# 5. Entrainer
model.fit(X_train, y_train)
print("Modele entraine!")
```

Classification Iris (3/3)

```
# 6. Predictions
y_pred = model.predict(X_test)

# 7. Evaluer
accuracy = accuracy_score(y_test, y_pred)
print(f"Precision: {accuracy * 100:.2f}%")

# Voir quelques predictions
print("\nExemples:")
for i in range(5):
    print(f"Reel: {y_test[i]}, Predit: {y_pred[i]}")
```

Résultat

Précision: 96-100% sur Iris!

Structure Standard Sklearn

```
from sklearn.quelconque import MonModele

# 1. Creer le modele
model = MonModele(parametres)

# 2. Entraîner
model.fit(X_train, y_train)

# 3. Predire
predictions = model.predict(X_test)

# 4. Evaluer
from sklearn.metrics import accuracy_score
score = accuracy_score(y_test, predictions)
```

Important

Même structure pour TOUS les algorithmes sklearn!

```
from sklearn.preprocessing import StandardScaler

# Normaliser (moyenne=0, std=1)
scaler = StandardScaler()

# Fit sur train, transform train et test
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Utiliser les donnees normalisees
model.fit(X_train_scaled, y_train)
predictions = model.predict(X_test_scaled)
```

Pourquoi?

Certains algorithmes sont sensibles aux échelles!

Exercice D'application

Votre Premier Modèle

- 1 Chargez le dataset Iris
- 2 Divisez en train (70%) et test (30%)
- 3 Entraînez un KNN avec $k=5$
- 4 Faites des prédictions
- 5 Calculez et affichez la précision
- 6 Affichez quelques prédictions vs réalité

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

① Comprendre le problème

- Classification? Régression? Clustering?

② Explorer les données

- `df.info()`, `df.describe()`, visualisations

③ Préparer les données

- Valeurs manquantes, normalisation

④ Entraîner un modèle

- Commencer simple (KNN, arbres...)

⑤ Évaluer et améliorer

- Métriques, validation croisée

Template Complet (1/2)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Charger
df = pd.read_csv('data.csv')

# 2. Explorer
print(df.head())
print(df.info())
df.hist()
plt.show()
```

Template Complet (2/2)

```
# 3. Preparer
X = df.drop('target', axis=1).values
y = df['target'].values

# 4. Diviser
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 5. Normaliser
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 6. Entrainer
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

# 7. Evaluer
y_pred = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
```

Documentation

- NumPy: numpy.org/doc
- Pandas: pandas.pydata.org/docs
- Matplotlib: matplotlib.org
- Scikit-learn: scikit-learn.org

Pratiquer

- Kaggle: datasets et tutoriels
- Google Colab: notebooks gratuits
- UCI ML Repository: datasets

Récapitulatif

Vous savez maintenant

- **NumPy**: calcul numérique rapide
- **Pandas**: manipulation de données
- **Corrélation**: interpréter les relations
- **Matplotlib**: visualisation
- **Scikit-learn**: modèles ML

Prochaine étape

Algorithmes de classification!